- 03 — Multi-Agent Graph
  - File: python/agentic/graph_v2.py
  - The Pipeline at a Glance
  - The Four Agents
    - 1. 📦 Data Loader
    - 2. 📊 Analyst
    - 3. 🔍 Code Reviewer
    - 4. 📝 Reporter
    - 👔 Manager (Quality Reviewer)
  - State Management
  - Graph Assembly
    - Wiring Diagram
  - Routing Logic
    - specialist_router(state) → str
    - after_tools(state) → str
    - manager_router(state) → str
  - Message Management
    - Constants
    - compress_messages(messages)
    - strip_tool_messages(messages)
    - build_focused_context(all_msgs, stage_name)
  - Tool Binding
  - Error Handling
  - Next

03 — Multi-Agent Graph

This is the brain of the system. The multi-agent graph orchestrates four specialist AI agents in a deterministic pipeline, with a quality manager reviewing each stage. Let's break it all down.

File: `python/agentic/graph_v2.py`

The Pipeline at a Glance

```
┌──────────┐     ┌──────────┐     ┌──────────────┐     ┌──────────┐
│  START   │──►  │  Data    │──►  │   Analyst    │──►  │  Code    │
│          │     │  Loader  │     │  (EDA, SPC,  │     │ Reviewer │
└──────────┘     └──────────┘     │  charts)     │     └──────────┘
                      │           └──────────────┘          │
                      ▼                  ▼                  ▼
                 ┌─────────┐        ┌─────────┐          ┌─────────┐
                 │ Manager │        │ Manager │          │ Manager │
                 │ Review  │        │ Review  │          │ Review  │
                 └─────────┘        └─────────┘          └─────────┘
                      │                  │                   │
                      │    ACCEPT ───────┘     ACCEPT ───────┘
                      ▼
                 ┌──────────┐
                 │ Reporter │──────► END
                 │ (writes  │
                 │  report) │
                 └──────────┘
```

Each specialist does its job, then the **Manager** reviews the output:

- **ACCEPT** → move to the next stage
- **REWORK** → send the specialist back to redo/fix (max 1 rework per stage)

The Four Agents

1. 📦 Data Loader

Role: Fetch data from PostgreSQL into memory.

Tools available: `query_mill_data`, `query_combined_data`, `get_db_schema`

What it does:

- Parses the user's question to figure out which mills and date range to load
- Calls `query_mill_data` for each mill (e.g., mills 1–12 for "all mills")
- Always computes `start_date` and `end_date` to filter at the SQL level
- Writes a brief summary: which mills were loaded, how many rows, date range

System prompt excerpt:

> *"You are the Data Loader. Your ONLY job is to call query_mill_data to load data. CRITICAL: ALWAYS compute start_date and end_date to filter at SQL level."*

Important rule: The Data Loader must NOT analyze data — only load it.

---

2. 📊 Analyst

Role: Perform statistical analysis and generate charts.

Tools available: `execute_python`, `list_output_files`

What it does:

- Calls `list_dfs()` to see what DataFrames are loaded
- Writes and executes Python code via `execute_python` for:
  - **EDA:** Distribution plots for Ore, PSI80, DensityHC, MotorAmp
  - **SPC:** Control charts with mean ± 3σ lines
  - **Correlations:** Heatmaps via `df.corr()`
  - **Anomaly detection:** Z-scores with threshold=3
  - **Downtime analysis:** Periods where Ore < 10 t/h
  - **Multi-mill comparisons:** Bar charts comparing mills side-by-side

Chart quality rules (from prompt):

- `sns.set_theme(style='whitegrid', font_scale=1.2)`
- Specific figure sizes per chart type
- All axes must have labels with units
- All charts must have descriptive titles
- Save with `dpi=150, bbox_inches='tight'`
- Always call `plt.close()` after each figure

---

3. 🔍 Code Reviewer

Role: Validate that the analysis produced correct outputs.

Tools available: `execute_python`, `list_output_files`

What it does:

1. Calls `list_output_files` to check what charts were generated
2. Reviews stdout from previous steps for errors
3. If there are **no chart files** (.png), calls `execute_python` to regenerate them
4. If charts exist and results look good, writes a validation summary
5. Does NOT regenerate charts that already exist

---

4. 📝 Reporter

Role: Write the final Markdown report.

Tools available: `list_output_files`, `write_markdown_report`

What it does:

1. Calls `list_output_files` to get exact chart filenames
2. Reads through ALL prior messages to extract statistics and findings
3. Calls `write_markdown_report` with the full report content

Report structure (from prompt):

- **Title** matching the analysis request
- **Executive Summary** — 3-4 sentences with key findings and actual numbers
- **Data Overview** — what was loaded, time range, row counts
- **Findings sections** — organized by analysis type, with real numbers
- **Charts** — every `.png` embedded using `![description](filename.png)`
- **Conclusions & Recommendations** — 3-5 specific actionable items
- Minimum 1000 words

👔 Manager (Quality Reviewer)

Role: Review each specialist's output and decide: ACCEPT or REWORK.

Not a stage itself — it runs between stages as a review step.

Evaluates:

- **Completeness:** Did the specialist do everything asked?
- **Quality:** Are charts well-formatted? Is the report detailed?
- **Correctness:** Are there errors or unreasonable values?

Responds with exactly one of:

- `ACCEPT: [brief reason]` — proceed to next stage
- `REWORK: [specific instructions]` — send back to fix

Special rules:

- Skips review for the Data Loader (always accepts)
- Max 1 rework per stage — if already reworked, forces ACCEPT
- Uses the base LLM without tool bindings (no tools)

State Management

The graph uses `AnalysisState`, which extends LangGraph's `MessagesState`:

```
class AnalysisState(MessagesState):
    current_stage: str           # "data_loader", "analyst", etc.
    stage_attempts: dict         # {"analyst": 1, "reporter": 0, ...}
```

- `messages` — The full conversation history (inherited from `MessagesState`)
- `current_stage` — Which specialist is currently active
- `stage_attempts` — How many times each stage has been attempted (for rework tracking)

Graph Assembly

The graph is built dynamically in `build_graph()`. Here's how it's wired:

```
STAGES = ["data_loader", "analyst", "code_reviewer", "reporter"]

# For each stage, create two nodes:
#   {stage}_entry  — sets current_stage in state
#   {stage}        — the actual specialist logic
```

```
# Plus shared nodes:
#   tools          — executes tool calls
#   manager_review — quality review between stages
```

Wiring Diagram

```
data_loader_entry ──► data_loader ──┬──► tools ──► (back to specialist)
                                    │
                                    └──► manager_review
                                              │
                                    ┌─────────┴──────────┐
                                    ▼                    ▼
                              ACCEPT:                REWORK:
                              analyst_entry           data_loader_entry
                                    │                 (re-run stage)
                                    ▼
                              analyst ──┬──► tools ──► (back to analyst)
                                        └──► manager_review
                                                  │
                                        ... and so on through all 4 stages ...
                                                  │
                                            (after reporter)
                                                  ▼
                                                END
```

Routing Logic

Three routing functions control the flow:

`specialist_router(state) → str`

After a specialist runs:

- If the LLM response has `tool_calls` → go to `"tools"` node
- Otherwise → go to `"manager_review"`

`after_tools(state) → str`

After tools execute:

- Find which specialist made the tool call (by checking `msg.name`)
- Return to that specialist (so it can see the tool results and continue)

`manager_router(state) → str`

After the manager reviews:

- If decision starts with `"REWORK:"` → go back to `{current_stage}_entry`
- Otherwise (ACCEPT) → advance to `{next_stage}_entry`
- If current stage is the last (reporter) → go to `END`

Message Management

The system carefully manages message history to stay within token limits:

Constants

```
MAX_TOOL_OUTPUT_CHARS = 2000     # Truncate tool outputs
MAX_AI_MSG_CHARS = 3000          # Truncate AI messages
MAX_MESSAGES_WINDOW = 14         # Keep only recent messages
MAX_SPECIALIST_ITERS = 5         # Max tool-call loops per specialist
```

`compress_messages(messages)`

Truncates long tool outputs and AI messages to stay within token budgets. Keeps only the last `MAX_MESSAGES_WINDOW` messages (plus the first one).

`strip_tool_messages(messages)`

Used for the manager (which has no tools). Converts:

- `ToolMessage` → `AIMessage` with `"[Tool result from X]: ..."` prefix

- `AIMessage` with `tool_calls` → plain `AIMessage` with text content

`build_focused_context(all_msgs, stage_name)`

The key optimization. Instead of sending the entire conversation to each specialist, it builds a focused context:

1. The original **user request** (HumanMessage)
2. A **compact summary** of prior stages (data loaded, analysis results)
3. Only **this stage's own messages** + tool results (for the tool-call loop)

This prevents context window bloat and keeps each specialist focused.

Tool Binding

Each specialist gets only the tools it needs:

```
TOOL_SETS = {
    "data_loader":    ["query_mill_data", "query_combined_data", "get_db_schema"],
    "analyst":        ["execute_python", "list_output_files"],
    "code_reviewer":  ["execute_python", "list_output_files"],
    "reporter":       ["list_output_files", "write_markdown_report"],
}
```

The LLM is bound to these specific tools per stage using `llm.bind_tools(stage_tools)`. This means:

- The Data Loader **cannot** run Python code
- The Analyst **cannot** write reports
- The Reporter **cannot** query the database

This enforces separation of concerns at the LLM level.

Error Handling

- **LLM errors:** If the LLM call fails, the specialist returns a message saying `"Error: ... Moving on."` and the pipeline continues.
- **Iteration cap:** If a specialist loops more than `MAX_SPECIALIST_ITERS` (5) times calling tools, it's forcibly moved to the next stage.
- **Manager errors:** If the manager review fails, it defaults to `ACCEPT` so the pipeline doesn't get stuck.
- **Rework limit:** Max 1 rework per stage prevents infinite loops.

Next

→ **04 — MCP Tools** — Deep dive into each tool